

- Display a ChatList
 - Each Chat should display an Avatar
 - We want to support high & low-res image here. Preview can be lower-res, upon tap we show higher-res.
 - Each chat should display a Preview which includes the chat title, a preview of the most recent message, and a timestamp of the most recent message.
 - Chat Title looks like its just the name of the contact/sender
 - Chat's should be sorted by timestamp, timestamp being the most recent message in that particular chat.

- If you click on a chat you should display the entire message history of the chat with most recent message on the bottom
- Message can contain arbitrary length text and multimedia images.
- Messages should show delivery receipts/state, so we need to support the states: Message Failed To Deliver, Message Delivered, Message In Route
- Be able to send messages (text & multimedia)

- ChatList and ChatView should quickly load for optimal UX
- ChatList and ChatView should have a smooth browsing experience (60 FPS)
- Offline Experience may be required which means:
 - Offline browsing of chats
 - Offline sending of messages. If poor network or no connection user can still send message, we have to indicate it didn't get sent and queue/retry message.
 - Offline Cache and Online syncing mechanism will be needed.
 - Optimize for slower networks & battery

- Chats are only single-user (chatting with one other user, no multi-user chat)
- We need not support additional media formats like audio and video
- No Typing Indicators
- No Push Notifications needed
- To keep things simple I am assuming images are attached and rendered always after text in a message. Otherwise messages need to become their own sort of rich text renderer and the message UI component becomes more complex. That's almost a design problem in and of itself.
- I am assuming the message text itself is cheap and not ridiculously large, but assuming image data can be rather large in bytes.

- HTTPS pagination endpoint to fetch chats
- Open up a WebSocket connection with the server to receive incoming new messages

On App Foreground of ChatList:

1. Fetch latest chats and render/display
2. Setup a Web-Socket connection to listen for new messages

display a boundedDeque/sliding window of [ChatPreview]'s
Below is what we'd serialize from the server response so its
the core data model, we'd transform into ViewModel's/Snapshot's for display.

```
PhotoAttachment {
    mediaKey: String
    size: CGSize
    imageURI: URL
}
```

Endpoint:

- <https://chatlist.app.com/get-chats?limit=50>
- <https://chatlist.app.com/get-chats?&cursor=34567&limit=50>

```

    },
    ....
  ],
  "media_keys": {
    "564_89": {

    },
    "4xY2_34": {

```

Endpoint: `wss://chatlist.app.com/message-preview/realtime`

```

Response:
/**
 * For an incoming message over our websocket in the ChatList
 * we could literally just reuse the data models and JSON structure from our
 * rest endpoints. I think it makes sense. The data for an incoming message
 * could roughly look like this.
 */
MessagePreview
{
  "messagePreview": {
    "chatID": "1234",
    "recentMessagePreviewText": ""
  },
  ...
}
}

```

The diagram illustrates a data transformation process from a 'Chat List' table to a 'Chat' table.

Chat List Table:

☐	Katy	Awesome!!	19:00
☐	Anoah	Thanks for the session today. It ...	12:30
☐	Max	You here?!	Sat
☐	Janie	Lamsters always pay their debts n)	10/04

Chat Table:

☐	Katy	Hey, what's up?		19:00
☐	Katy	Check this out		13:11
☐	Andrey	Awesome!!		19:00

An arrow points from the 'Chat List' table to the 'Chat' table, indicating a transformation process. The transformation involves selecting specific rows from the 'Chat List' table and mapping them to the 'Chat' table, where the 'message' column is split into 'message' and 'image' columns, and the 'timestamp' column is split into 'time' and 'date' columns.

- HTTPS Pagination endpoint to fetch messages via HTTP GET

Sending Messages (Little Tricker):

- We will have a `message_create` endpoint via HTTP POST. The idea here is this just creates a message on the backend but hasn't started delivery yet.
- We will have a `message_media_upload` endpoint. Again through HTTP Post

Receiving Realtime Messages:

- Open up a new WebSocket to the current chat
- Delivers any new messages for current chat (full fledged Message objects)
- Delivers RealTime Message status updates for sent/created messages.

Data Model:

- * when opening a chat ultimately we are displaying a `BoundedDeque` of `[Message]`'s for the current chatID
- * We will likely need to display some Chat Metadata (at the top) of the ChatView so we also have a `ChatDescriptor` object

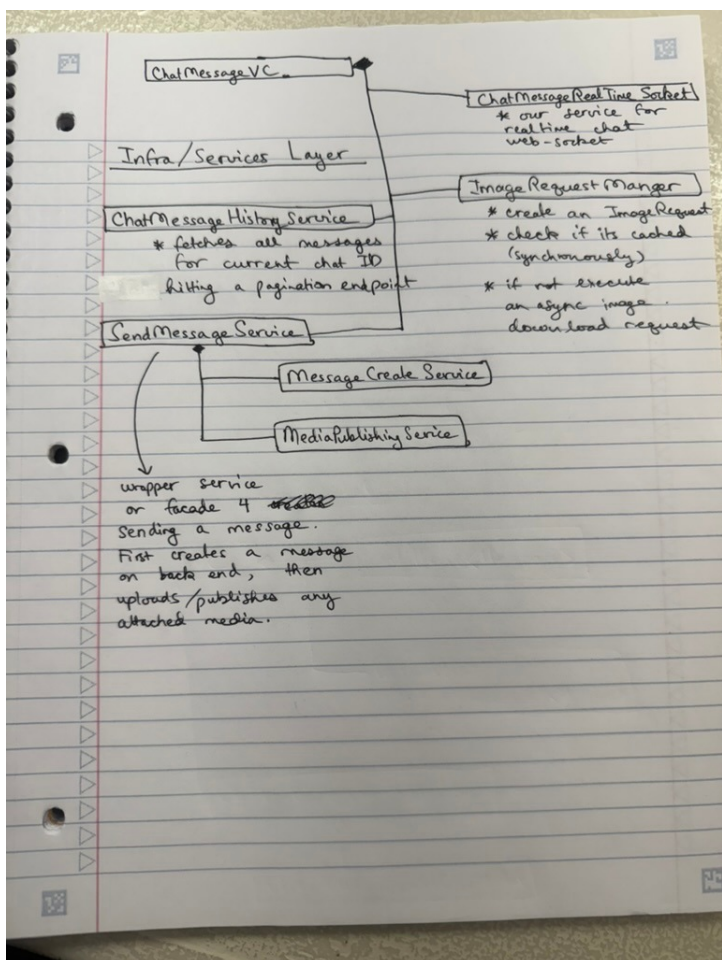
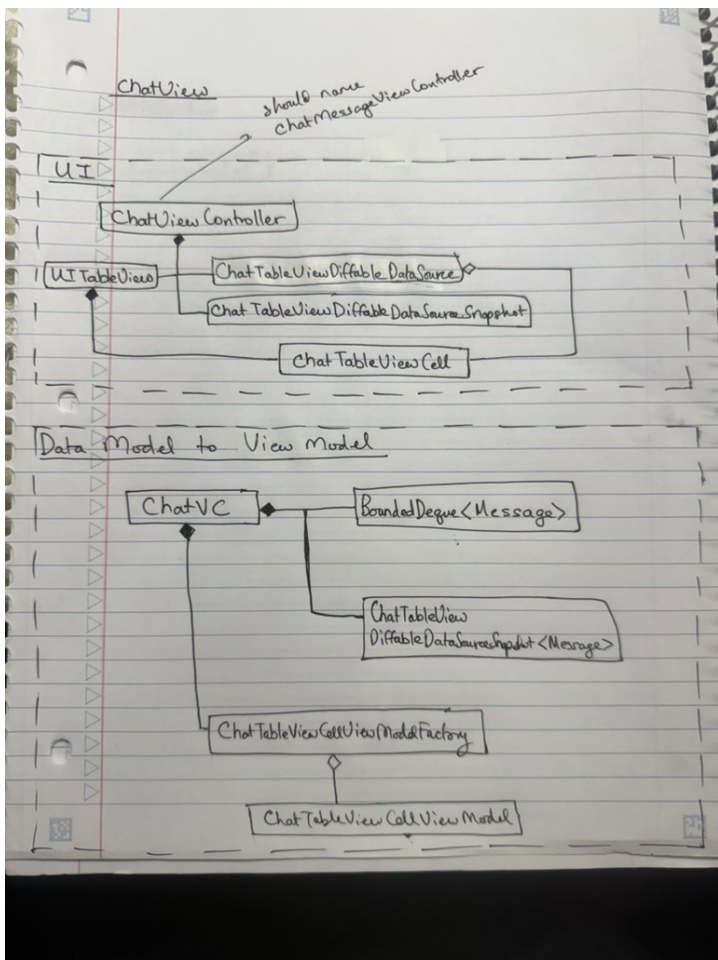
```
User {
  username: String
  userID: String
  profileImageAttachment: PhotoAttachment
  contactInfo: Contact
}
```

```
Contact {
  phoneNumber: String
  email: String
  address: String
  ....
}
```

```
ChatDescriptor {
  chatID: String
  peerOne: User
  peerTwo: User
}
```

```
MessageDeliveryStatus {
case messageCreated
case messageDeliveryInFlight
case messageDelivered
case messageCreationFailed
case messageDeliveryFailed
}
```

```
Message {
  sender: User
  recipient: User
  messageText: NSAttributedString
  mediaAttachments: [PhotoAttachment]
  messageID: String
  timestamp: Date
  deliveryStatus: MessageDeliveryStatus
}
```



Offline Experience and Client Server Sync

chatList VC

Details:

- Obviously all message data will be stored on client in a database, relational SQL makes the most sense so we can construct a index on <message_id, timestamp> and <chat_id, timestamp> for both the ChatList and ChatView (chat thread)
- Taking a step back the first thing to understand is each time we fetch in the chatList we get a batch of chats along with cursor tokens for next and previous.
- When new data comes in either over WebSocket or when we fetch the latest data using a cursor token remember each chatPreview has a chatID. This is where a difableDataSource comes in handy, since the backing data model (snapshot) is identity based, we can remove the previous item from our snapshot with the same chatID and insert the new item at the top, effectively "updating" the state of our chat.

Online Offline Experience for ChatList

1. Client fetches data from DB first. The current cursors are stored in the DB as well prevToken and nextToken
2. Render chatList data from DB
3. In parallel kick off a fetch for the latest chat data using the prevToken.
4. Server will return the latest 50 results/updates. If there are more updates server will return another prevToken
5. We update the chatList with the latest data (modifying the snapshot) and reloading the data.
6. Whether we navigate the user to the top of the list or keep them where they are with a little "scroll up for recent chats notification" this can be studied and is more of a UX specific issue.
7. After modifying our chatList and snapshot (in-memory data model), we reconcile the database and update the records for each chatID that is modified.
 - An alternate design is fetch the latest and update the DB first. Then announce those changes to update the data model and then the snapshot and UI. This way the DB always acts as the source of truth and the UI is just reacting to what's in it. That might be cleaner but I am not sure how to announce a change and update the data model if the DB changes.
 - The takeaway is instead of a "diff" API, get the latest diff chat changes, I will stick with my pagination approach to keep things simple. It's also not clear how a diff API handles the case if there are a HUGE amount of changes. Atleast in the pagination model its predictable.

A Note about chatLIST DB TTL's

- We can have a per chatPreview record TTL
- We will also have a more global chatList TTL
- The global chatList TTL is used to indicate if the data is so old/stale its just a better UX to wipe the DB clean and reload the chatList completely from the server response.
- Per Item TTLs can be cleaned up periodically by our Manager object/facade writing and reading to the DB.
- TTL will be a column we construct an index on as well, likely index will be <chatID, TTL> for tie breaks
- The idea behind TTL's is to keep the disk footprint down.

ChatView (Or Chat Thread)

- A lot of the online vs. offline pagination catchup stuff is going to be the same as above, the main thing for this view is enqueueing messages.

- For each message that fails to get sent or delivered write those messages to a new DB Table called pendingMessages. There can be a retry count associated with each message.

- On app foreground directly retry any PendingMessages in the table, Send messages via HTTP Post endpoint, don't rely on WebSocket.

Feedback on the Problem after reviewing Solution:

